ELEC3441: Computer Architecture
Second Semester, 2023–24

Lab 1

`Objective: To practice and revisit program compilation and disassembly.`

1 Compilation

Compiler is essential to bridge the gap between programs written in high-level languages such as C and the processor. In this question, you will experiment with the basics of the GNU C compiler (`gcc`) that is ported to compile for the RISCV processor.

1.1 Preliminaries You will perform this exercise in the course Docker environment and use Visual Studio Code (VS Code) to edit files and access the container terminal. The Docker image contains a pinned version of the `riscv-gnu-toolchain`, including tools such as `gcc`, `objdump`, and `gas`. These tools produce RISC-V instructions even when Docker is running on a computer with a different processor architecture – a process called **cross compilation**.

To conform to the GNU naming convention for cross compiler, you will find that the tools we are using has a rather long prefix in the name, such as:

- `riscv32-unknown-elf-gcc`
- `riscv32-unknown-elf-objdump`
- `riscv32-unknown-elf-gas`

Before starting, install the following software:

- Docker Desktop
- Visual Studio Code
- The Microsoft **Dev Containers** extension for VS Code

In a Linux terminal, press `<TAB>` to complete a partially typed command or filename. This can save a considerable amount of typing.

1.2 Start the Docker Environment Start Docker Desktop and wait until the Docker engine is running. Open a terminal on your own computer, change to the directory containing the course `manage` script, and run:

```
./manage build
./manage verify
./manage up
```

The `build` command is required only the first time, or after the course image has been updated. The `verify` command checks that the required tools are available. The `up` command starts the persistent course workspace. It also prints a browser-desktop address used by GUI-based tools in later labs; Lab 1 can be completed entirely in VS Code.

1.3 Open the Lab in VS Code In VS Code, open the Command Palette and select **Dev Containers: Attach to Running Container...** Select `elec3441-lab`. When the new VS Code window opens, select **File** → **Open Folder...** and open:

```
/workspace/labs/lab1
```

1.4 Verify the Workspace Select **Terminal** → **New Terminal** and verify the environment:

```
pwd
ls
riscv32-unknown-elf-gcc --version
```

The source files are directly under `/workspace/labs/lab1`; there is no additional `compilation` subdirectory. Files saved under `/workspace` persist when the container is stopped and restarted. These working copies are editable; the originals remain safely stored in the Docker image.

1.5 Compile the file `ArrayAccu.c` to RISC-V assembly code using:

```
riscv32-unknown-elf-gcc -O1 -S ArrayAccu.c -o ArrayAccu.s
```

The switch `-S` tells `gcc` to produce human-readable assembly code instead of a machine-readable binary. The switch `-o` tells `gcc` to store the output in `ArrayAccu.s`. Finally, `-O1` (capital letter O) enables level-one optimization. This lab compares `-O0`, which disables optimization, with `-O1`.

Look at the content of `ArrayAccu.s` and answer the following questions:

1. The program starts running with the C function `main`. Where is `main` defined in `ArrayAccu.s`?
2. Search through `ArrayAccu.s`, where is `main` being called?
3. The `ArrayAccu()` function is called within `main`. Look at the code before the function is called and the code **inside** `ArrayAccu()`. Where are the input arguments and return value stored?
4. Can you see how the return address `ra` is used?

1.6 Check Yourself

Make sure you have completed the steps before coming to the lab. Be prepared to show your work during the lab.

1.7 Branches Compile `branch1.c` and `branch2.c` at optimization levels `-O0` and `-O1`:

```
for name in branch1 branch2; do
    riscv32-unknown-elf-gcc -O0 -S ${name}.c -o ${name}_0.s
    riscv32-unknown-elf-gcc -O1 -S ${name}.c -o ${name}_1.s
done
```

Read through the compiled code and answer the following questions:

1. Does the compiler implement the two branches differently? How are they different?
2. How many static RISC-V instructions are present in each function? Do not count labels or assembler directives.
3. How does the generated code differ between `-O0` and `-O1`?

1.8 Loops Compile `loop1.c` and `loop2.c` using `-O0` and `-O1`, similar to the preceding exercise. Then answer the following questions:

1. How does the compiler implement the two different loops?
2. How many instructions are needed for the two loops?
3. How does the generated code differ between `-O0` and `-O1`?
4. What work does loop fusion remove, and why can the compiler no longer delete the loops entirely?

1.9 If-then-else vs. Switch Compile `branch.c` and `case.c` using `-O0` and `-O1` as before, and answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1.10 Function Calls Compile and compare the two files `funcall1.c` and `funcall2.c`, then answer the following questions:

1. How does the compiler implement the following two code segments differently?
2. How does the generated code differ between `-O0` and `-O1`?

1.11 Submission Submit short answers for Sections 1.8 through 1.10 on Moodle.